VIA University College



Programming 3

Autumn 2026

$whoami

Knowledge

- The students will be able to
  - describe various distributed system types
  - explain various distributed system architectures
  - explain various distributed communication methods
  - explain the use of contracts in service calls
  - explain peer-to-peer systems
  - explain desired qualities of distributed systems

Skills and Competences

Skills

- The students will be able to:
  - use various distributed communication methods for direct and indirect communication
  - argue the choice of middleware for a given distributed system
  - compare peer-to-peer systems with client/server systems
  - design and implement robust distributed systems

Competences

- The students will be able to:
  - design the architecture of a distributed system using known architectural patterns
  - design and implement a distributed system on different platforms using various middleware

Course Overview/Semester plan

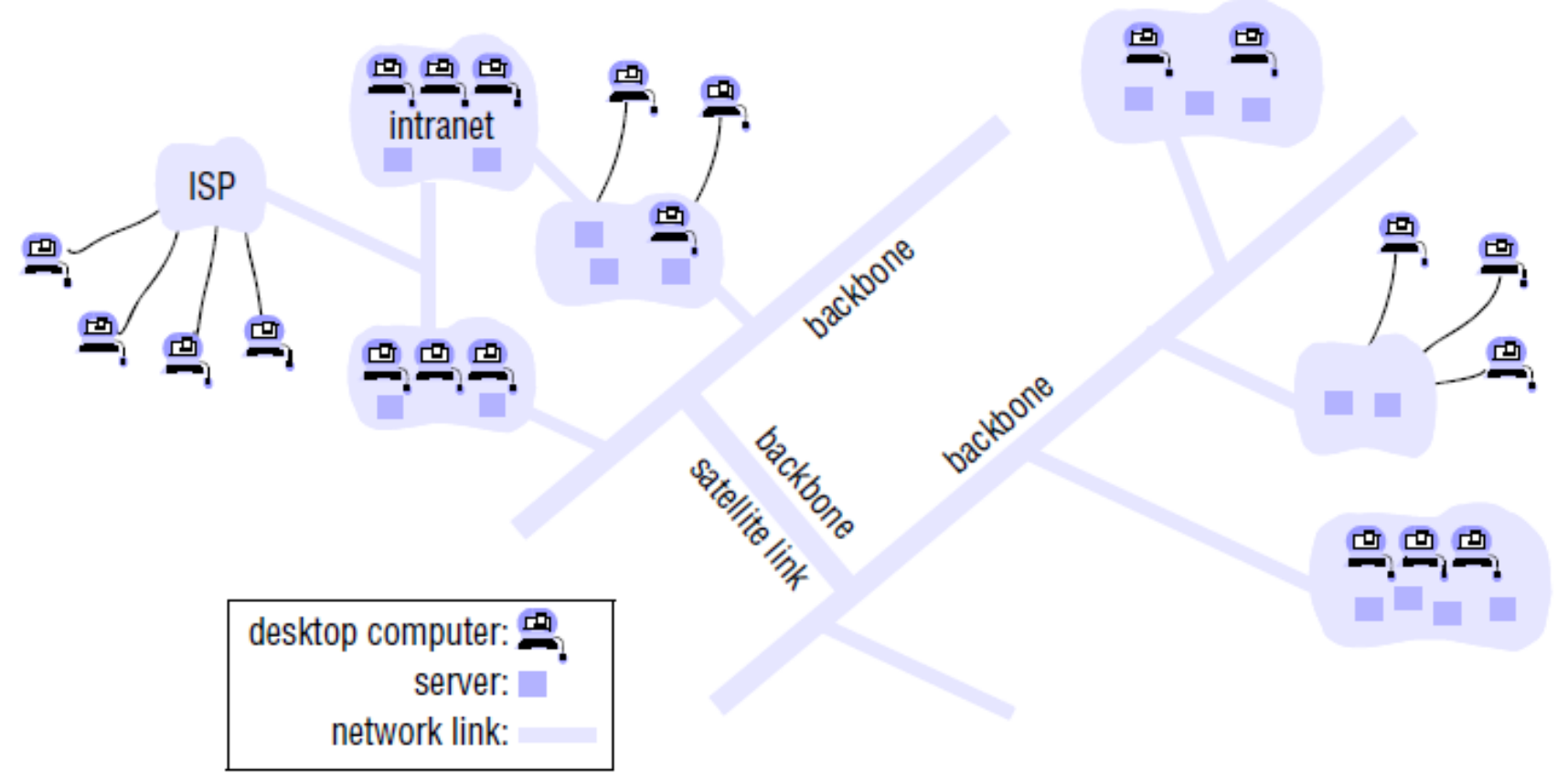
Week	Date	Topics	Keywords	Literature
36 – 1	Aug 31	Introduction to Distributed Systems, Architectural qualities Distributed System Models **Course Assignment Part 1**	Heterogeneity, Security, Scalability, Concurrency, Transparency, Quality of Service, N-Tier, Microservices, C4 Model, CAP theorem	Ch. 1 Ch. 2.1-2.3
37 – 2	Sep 7	Introduction to Spring Framework, Spring boot fundamentals, Maven project structure	Maven, Spring Boot	
38 – 3	Sep 14	Remote Procedure Call & gRPC, Build automation and management **Course Assignment Part 2**	Protocol Buffers gRPC API Contract-based protocols	
39 - 4	Sep 21	Distributed Objects and Components, Dependency Injection, DI Containers	Spring IoC, Error/Failure Handling	Ch. 8
40 – 5	Sep 28	Introduction to Web Services **Course Assignment Part 3**	HTTP, REST	Ch. 9
41 – 6	Oct 5	RESTful Web Services, Microservices	URI, JSON, REST APIs, RestClient, WebClient	
42	Oct 12	Autumn Break		
43-7	Oct 19	Data Access/Database Communication	Spring Data JPA, Repository pattern	
44-8	Oct 26	Indirect Communication 1 **Course Assignment Part 4**	DSM, JMS Publish-Subscribe	Ch. 6
45-9	Nov 2	Indirect Communication 2	Message Broker, Publish-Subscribe with RabbitMQ, AMQP	
46-10	Nov 9	Distributed Transactions, Two-Phase Commit	ACID, 2PC	Ch. 17
47-11	Nov 16	Peer-to-peer Systems Routing	Routing Overlays	Ch. 10
48-12	Nov 23	Exam info	Exam information	

PRO3 Examination

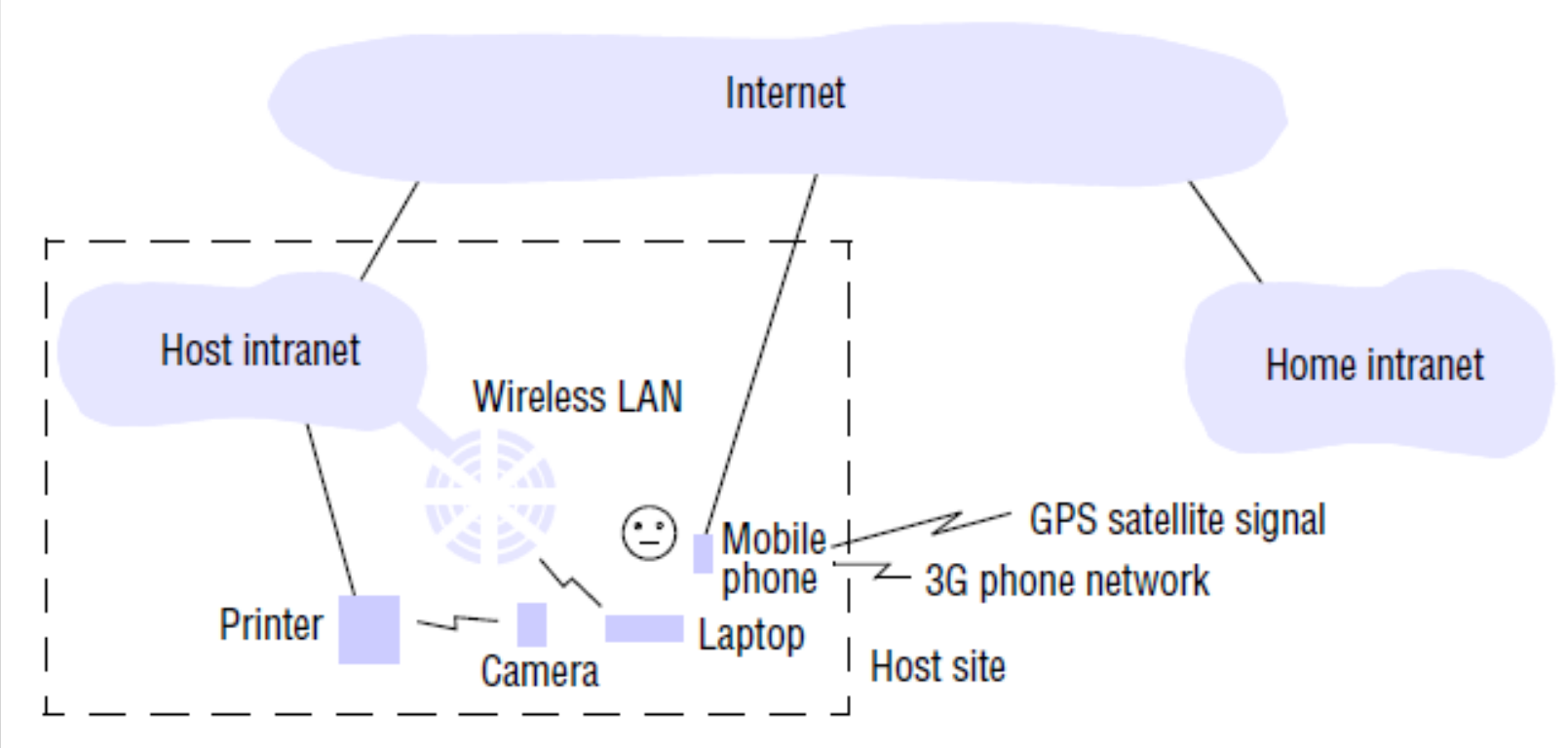
- Attendance (≥ 75%)
- 20 minutes individual oral examination
- Individual oral examination without preparation covering mandatory course assignment and theory covered in the course.
- You'll explain the theory behind a distributed system and implementation of some concepts of distributed systems.



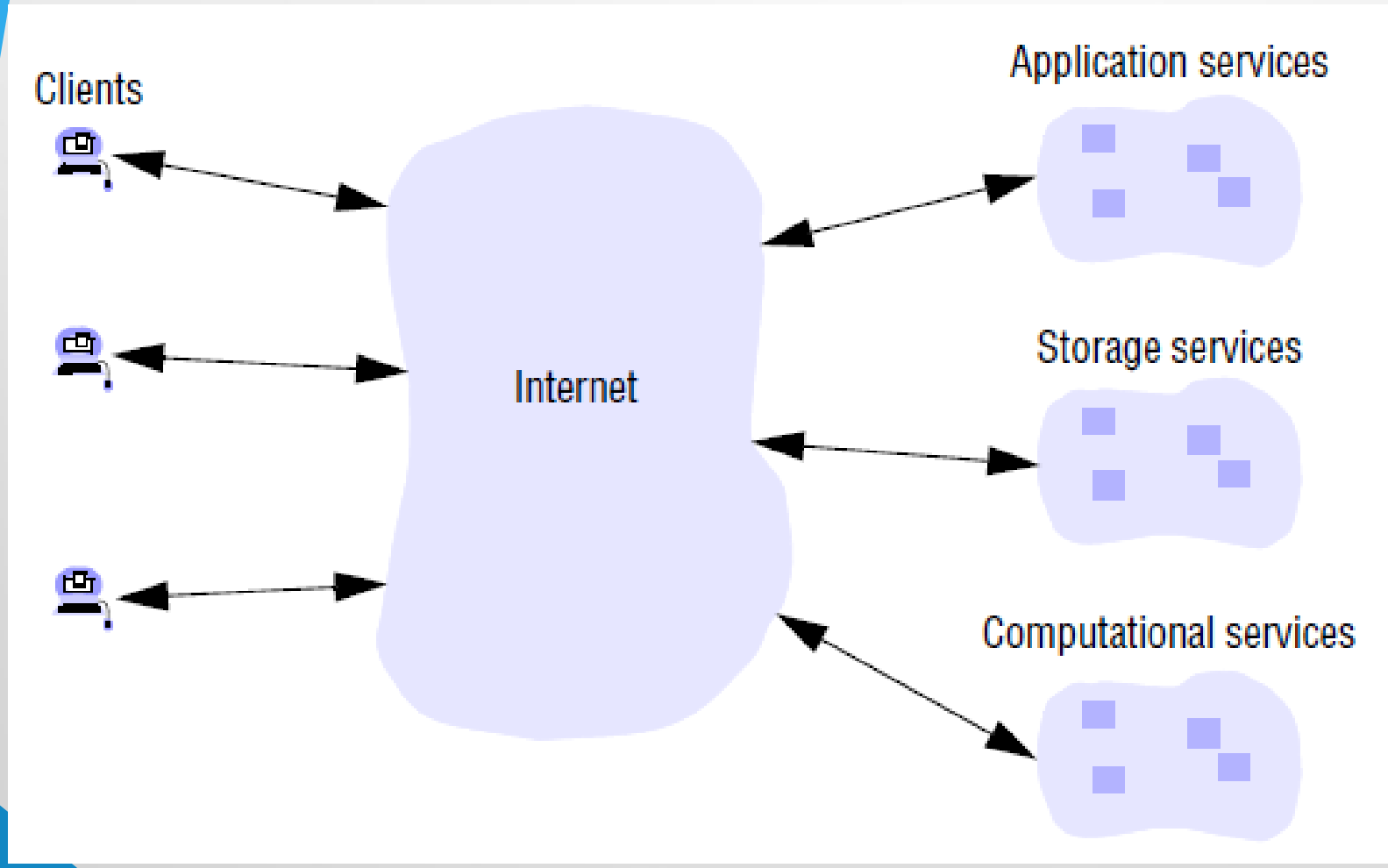
A Typical Portion of the Internet



Portable and handheld devices in a distributed system



Cloud Computing



What are Distributed Systems?

A *distributed system* is one in which components located at networked computers communicate and coordinate their actions only by passing messages.

False Assumptions Vs Unique Properties

False assumptions about a distributed application

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Properties unique to distributed systems

- ✓ reliability
- ✓ security
- ✓ heterogeneity
- ✓ topology of network
- ✓ latency and bandwidth

- ✓ transport costs
- ✓ administrative domains

Architectural Problems

- Heterogeneity
- Security
- Scalability
- Failure Handling
- Concurrency
- Transparency
- Quality of Service
  - Reliability
  - Performance
  - Maintainability
  - Consistency
  - Availability

Blue = typical non-functional requirements

Heterogeneity

- A system is heterogeneous when different platforms and languages are used in different parts of the system
- Architectural impact
  - A heterogeneous system is best avoided whenever possible because it can be difficult to maintain, the same logic is repeated many places in the system, and it can be difficult for different parts of the team to communicate.
- Technological impact
  - The technologies that can handle heterogeneous systems are more flexible, but sometimes also more complex.
  - Beware of making too much complexity to cater for the heterogeneity that you don't need.

Scalability

- Scalability is the ability to handle increased load by increasing the number of servers (horizontal scaling, preferred) or by upgrading a server (vertical scaling). Preferably the scaling should be linear, so that a doubling of the load can be handled by a doubling of the number of servers.
- Architectural impact:
  - Avoid $O(n^2)$ algorithms – in particular, remember indexes in the database
  - Avoid bottlenecks – components that all communication has to pass through.
- Technological impact:
  - See "Performance"

Failure Handling

- The holy grail of system development. In practice, difficult to do on high-level transactions.
- Architectural impact:
  - The basic tool of the architect is redundancy: If one component (typically a server or a disk) fails, have another take over.
  - Another way of having redundancy is to re-try when things don't work.
- Technological impact:
  - Several basic technologies have redundancy built-in.
  - Examples: TCP, RAID5, various databases.

Concurrency

- Dealing with concurrency is a balancing act: We want to have everything running in parallel, but we also want to restrict simultaneous access to shared resources.
- Architectural impact:
  - Several patterns deal with keeping the locking of resources to a minimum while still allowing parallelism. E.g. optimistic locking.
  - Some patterns try to avoid locking altogether. E.g. event sourcing.
- Technological impact:
  - Many technologies come with their own flexible locking mechanisms. Notably databases.

Transparency

- A transparent solution is one in which it makes little difference whether a system is distributed or local.
- Architectural impact:
  - It is controversial whether this is actually a desirable goal to strive for from an architectural point of view, since there will always be a difference between distributed and local systems.
- Technological impact:
  - There is rather large difference in how transparent various communication protocols are.

Reliability

- Reliability is strongly connected with availability and failure handling.

Performance

- Usually stated like a non-functional requirement: "The system must respond before 2 seconds 95% of the time."
- Architectural impact:
  - You can either try to spread the workload to different computers or just throw some meatier computers at the problem. The latter is also known as "Kill It With Iron" (KIWI).
  - Another important factor is to minimize the network communication.
  - Important to have actual measurements before trying to optimize.
- Technological impact
  - There's no shortage of companies who claim their products help with this. Again: Actual measurements.

Maintainability

- Not quite an objective quality, but important none-the-less.
- Architectural issues:
  - Design both objects and components with high cohesion and low coupling
  - Make a solid domain model for the problem
- Technical issues:
  - Some tools can help here.
  - There is no great difference between the technologies from the course in this respect.

Availability

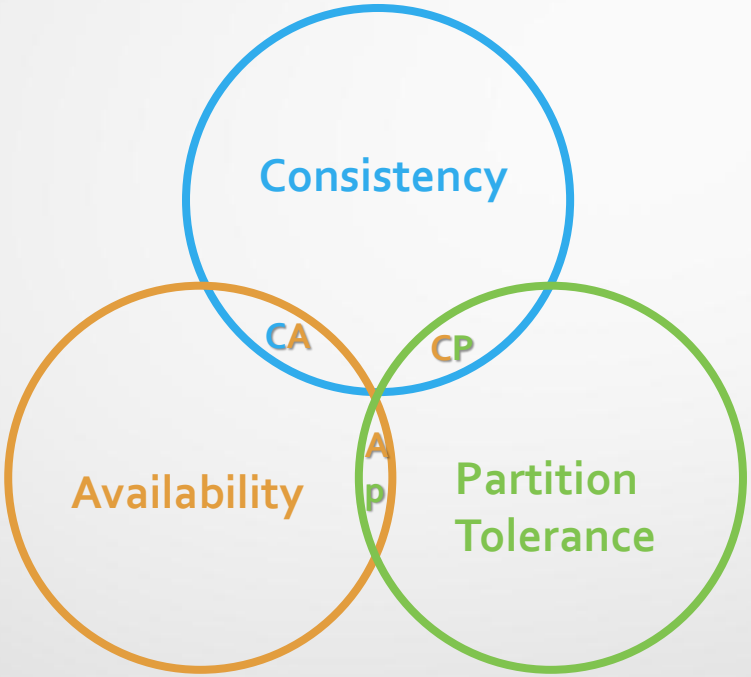
- Availability is about the system being responsive as often as possible.
- Architectural impact:
  - Redundancy helps a lot here, as does having the servers close to the user.
  - Several patterns are made directly for dealing with this problem.
- Technological impact:
  - Some of the patterns mentioned above are implemented directly in specialized software.
  - There is little difference in the standard software as far as this quality is concerned
- ***High availability is incompatible with high consistency***

Consistency

- Consistency in this context means that you will get the same result no matter where you use the system from.
- Architectural impact:
  - This, too, is the topic of a great number of patterns. Your entire design will typically be based around this.
- Technological impact:
  - This is intimately connected to the use of databases.
- ***High consistency is incompatible with high availability***

The CAP Theorem

- **Trade-offs** that must be made when designing distributed systems



The CAP Theorem

- **Trade-offs** that must be made when designing distributed systems
  - **Consistency.** All reads receive the most recent write or an error.
  - **Availability.** All reads contain data, but it might not be the most recent.
  - **Partition tolerance.** The system continues to operate despite network failures (ie; dropped partitions, slow network connections, or unavailable network connections between nodes.)
- *The CAP theorem states that it is not possible to guarantee all three of the desirable properties — consistency, availability, and partition tolerance at the same time in a distributed system with data replication.*

Understanding Distributed Systems



- What are the communicating entities?
- What communication paradigms are used?
- What roles and responsibilities do entities have?
- How are entities mapped onto physical resources?

… more details in the coming sessions